Coding Assignment 3: SLAM and Motion Planning

Yuer Tang

1 Part A: EKF-SLAM Trajectory and Uncertainty

1.1 A1. Path Design: Which Trajectory is Better for SLAM?

I tested two different control strategies in the same 15-landmark environment:

- **Path A (Greedy Forward):** L-shaped sweep. Straight east for 150 steps, turn left, straight north for 150 steps, turn left, then west. Each landmark is observed once or twice.
- **Path B (Looping Revisit):** Heads northeast, then enters tight circular orbits (ω=0.25 rad/s), translates, and does a second orbit. Landmarks near the orbit centres are observed 4–8 times.

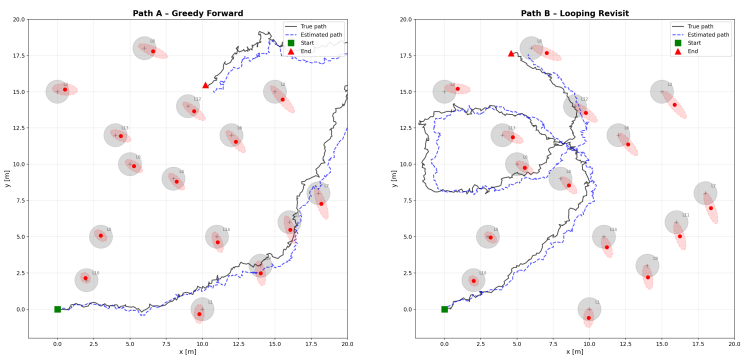


Figure 1: True vs. estimated path. Pink/red ellipses = 2σ landmark uncertainty.

Table 1: A1 comparison (500 steps).

	Greedy	**Loop**
Final pose err	0.669 m	1.204 m
Mean pose err	0.837 m	0.775 m
Final pose unc	0.572 m	0.607 m
Avg LM unc	0.215	0.217
LMs observed	15/15	15/15

Greedy Forward has a lower *final* pose error (0.669 vs. 1.204 m), but Looping has a lower *mean* error (0.775 vs. 0.837 m). This is because the looping path keeps re-observing landmarks, which triggers EKF corrections that pull the estimate back toward truth. The greedy path drifts during long straight segments but happens to end near landmarks that anchor the final estimate. Both paths observe all 15 landmarks, so final landmark uncertainties are nearly identical (≈ 0.215).

Uncertainty ellipse evolution. fig. 2 shows map snapshots at steps 50, 150, 250, 350, and 500. Each dot is an observed landmark. The colour encodes uncertainty: blue means confident, red means uncertain. The ellipse around each dot is the 2σ confidence region. A large ellipse means the EKF is unsure where the landmark is.

Under Path A, ellipses shrink when a landmark is first observed but stay the same size after that, because the robot moves on and never returns. Under Path B, landmarks near the orbit centre are re-observed on every loop. Each re-observation triggers an EKF update where the Kalman gain uses the cross-covariance to correct both the robot pose and landmark position. The ellipses visibly collapse from large red shapes (step 50) to tight blue dots (step 350). This is the **loop-closure effect**, the most important mechanism in EKF-SLAM for reducing uncertainty.

1.2 A2. Noise Sensitivity

I used Path B for all noise experiments because it is the stronger baseline.

Pose accuracy: Increasing process noise from Low to High raises pose error from 0.278 to 1.857 m (6.7×). Increasing measurement noise over the same range only raises it from 1.263 to 1.351 m (1.1×).

Map accuracy: Landmark position error follows the same pattern. Process noise: 0.309 to 1.124 m (3.6× worse). Measurement noise: 0.793 to 0.875 m (1.1×). Process noise corrupts the robot pose, which in turn corrupts every landmark update that depends on it.

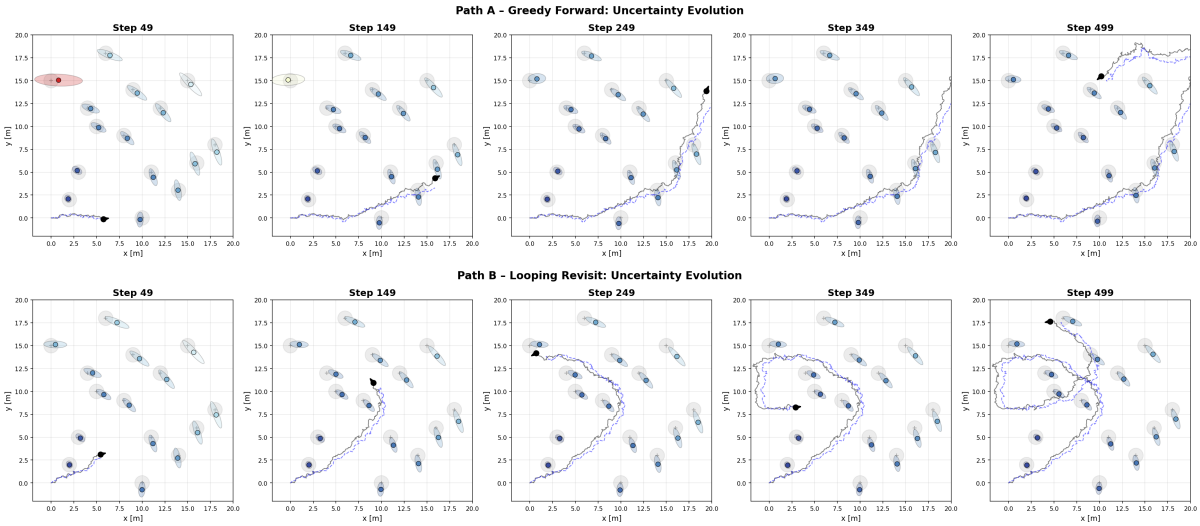


Figure 2: Uncertainty snapshots. *Top:* Path A, ellipses stay fixed after first observation. *Bottom:* Path B, ellipses collapse with repeated re-observation.

R Level	Pose Err	LM Err	Pose Unc	LM Unc	Q Level	Pose Err	LM Err	Pose Unc	LM Unc
Low	0.278	0.309	0.452	0.167	Low	1.263	0.793	0.453	0.126
Med	1.204	0.772	0.607	0.217	Med	1.204	0.772	0.607	0.217
High	1.857	1.124	0.861	0.384	High	1.351	0.875	0.876	0.457

Table 2: Noise sensitivity (all in metres). Left: varying process noise $\boldsymbol{R}$ (Q fixed at Med). Right: varying measurement noise $\boldsymbol{Q}$ (R fixed at Med). Pose/LM Err = position error. Pose/LM Unc = $\sqrt{\text{trace}(\boldsymbol{P})}$.

Overall uncertainty: Pose uncertainty grows from 0.452 to 0.861 m under high process noise, and from 0.453 to 0.876 m under high measurement noise. Here the effect is similar because the EKF correctly inflates its uncertainty estimate in both cases. Landmark uncertainty grows more under measurement noise (0.126 to 0.457) than process noise (0.167 to 0.384), because noisy sensors directly degrade the observation model used to update landmarks.

Process noise is far more damaging because it corrupts the robot state at every predict step (500 total), so drift accumulates between observations. Measurement noise only degrades the quality of individual corrections and gets averaged out by repeated observations.

Does the looping path hold up? Even under high process noise, the looping path stays at 1.857 m, compared to what would be unbounded drift without any observations. Under high measurement noise it stays at 1.351 m. The looping strategy gives built-in protection because re-observing landmarks keeps correcting the estimate. The path does not break down under any noise level tested.

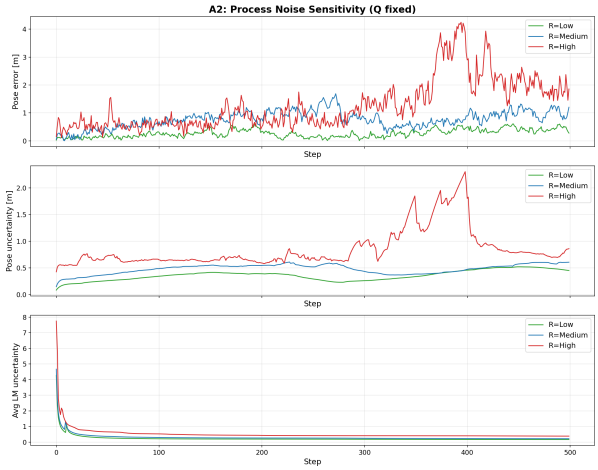
2 Part B: PRM vs RRT Planning Comparison

2.1 B1. Start/Goal Scenarios

- **Scenario 1 (Open):** Obstacle radius 0.8, start= $[1, 1]$, goal= $[19, 1]$. Wide gaps between obstacles.
- **Scenario 2 (Constrained):** Obstacle radius 1.0, start= $[1, 1]$, goal= $[17, 17]$. Tight passages.

Why PRM suits open environments. In open space, uniform sampling easily covers the free space, producing a dense roadmap (fig. 4a, left). A* then finds a near-optimal path through this graph. The upfront cost of building the full roadmap is worthwhile because the resulting path quality is high, and the roadmap can be reused for multiple queries.

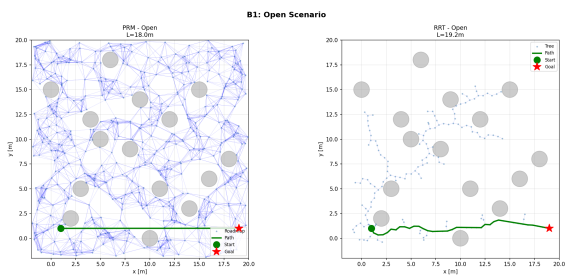
Why RRT suits constrained environments. In tight passages, PRM wastes many samples in open areas while struggling to place enough nodes inside narrow corridors. RRT grows incrementally from the start toward the goal, so it only explores where needed (fig. 4b, right). Its step-by-step extension naturally threads through gaps without requiring dense coverage of the whole map. RRT is also much faster, which matters in constrained settings where you may need to replan.



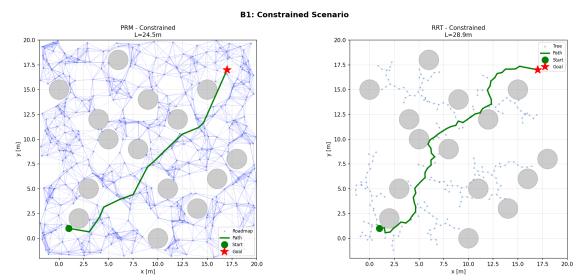
(a) Process noise sweep (***Q*** fixed).

(b) Measurement noise sweep (***R*** fixed).

Figure 3: A2 noise sensitivity. Process noise (left) is far more damaging than measurement noise (right).



(a) Open (radius 0.8). PRM builds a dense roadmap; RRT grows toward the goal.



(b) Constrained (radius 1.0). RRT threads through narrow gaps with fewer nodes.

Figure 4: B1 scenarios. Blue = roadmap/tree edges, green = solution path. Each panel shows PRM (left) and RRT (right).

2.2 B2. Metrics Comparison (Constrained, N=30)

Metric	PRM	RRT
Success rate	100%	100%
Avg path length	24.48 m	28.72 m
Avg time	1.762 s	0.068 s
Avg nodes	500	217

Table 3: B2 metrics (constrained scenario, 30 trials).

Both achieve 100% success. PRM produces 15% shorter paths because A* finds the optimal route on the roadmap. RRT is 26× faster because it does not build a full graph. The RRT paths zigzag visibly (fig. 4b, right) because each extension steps toward a random sample. PRM's A* finds the shortest graph path through dense collision-free nodes. This is the core trade-off: PRM gives better paths but costs more time. RRT is fast but paths need post-processing.

2.3 B3. Parameter Studies

fig. 5 shows path length (left) and planning time (right) for all three parameter sweeps. Success rate is 100% in all cases except RRT with zero goal bias (95%).

PRM: n_samples. Path length decreases from 25.82 m (50 samples) to 24.27 m (800), a 6% improvement. Most of the gain happens between 50 and 200; going from 200 to 800 only saves 2.4%. Planning time grows from 0.28 s to 2.90 s because building the roadmap is $O(n^2)$. Beyond 200–400 samples, the cost is not worth the diminishing path improvement.

RRT: step_size. Path length increases slightly from 28.40 m (0.2) to 29.59 m (2.0) because larger steps overshoot and create coarser paths. Planning time drops from 0.30 s to 0.016 s because fewer iterations are needed. In narrow passages, large steps risk jumping over valid corridors. For this environment, 0.5 is a good balance.

RRT: goal_sample_rate. This is the most impactful parameter. With zero bias, success drops to 95% and planning takes 1.62 s because the tree explores randomly. Even 0.05 bias fixes success to 100% and cuts time to 0.08 s. Higher bias (0.5–0.7) shortens paths from 28.5 m to 26.8 m by steering growth toward the goal. High bias works here because the direct route is mostly clear. In more cluttered environments, high bias would hurt because the tree would repeatedly try to extend through blocked areas.

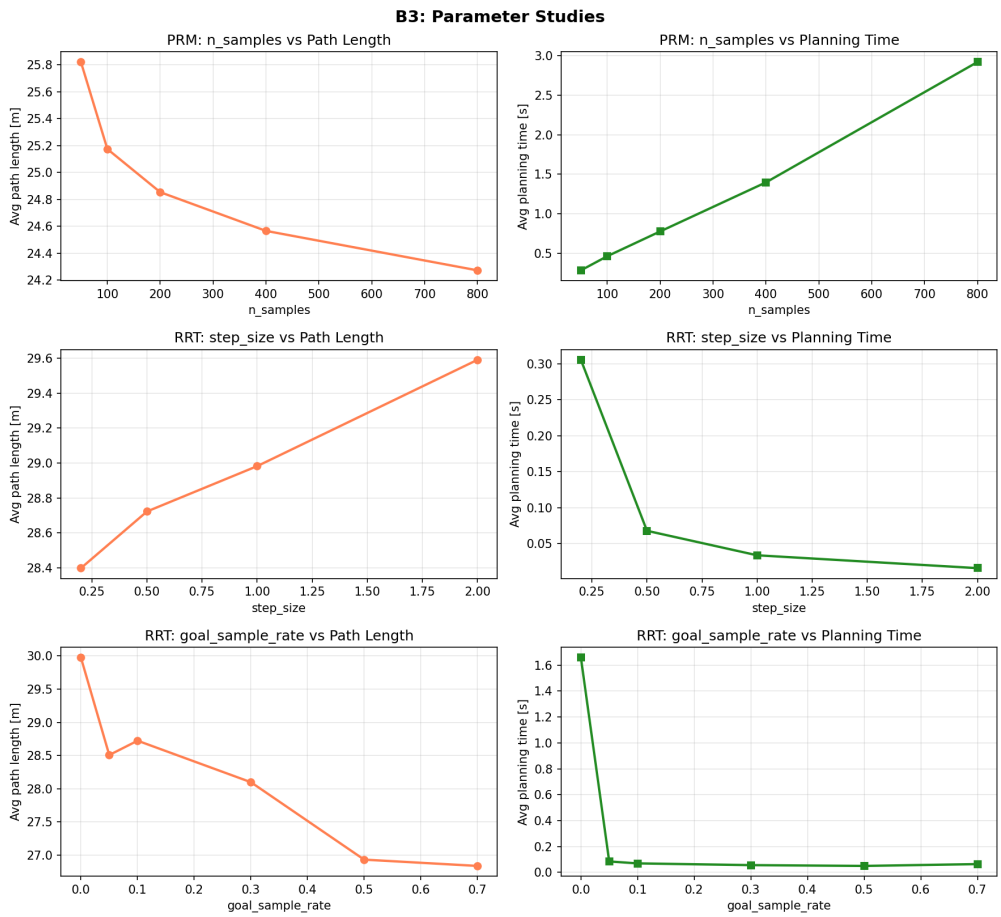


Figure 5: B3 parameter studies. Left: path length. Right: planning time. Top: PRM n_samples. Middle: RRT step_size. Bottom: RRT goal_sample_rate.

3 Part C: Improving PRM and RRT

What I changed. I added a **path shortcutting** post-processing step to both planners. After a path is found, the algorithm randomly picks two non-adjacent waypoints w_i, w_j and checks if the straight line between them is collision-free. If so, all intermediate waypoints are removed. This is repeated for 200 iterations. The motivation is that PRM paths follow discrete graph edges that may detour around sampled nodes, and RRT paths consist of many tiny incremental steps (≤ 0.5 m each) that zigzag toward the goal. Shortcutting replaces these redundant segments with direct lines.

Experiment. I tested on the constrained scenario from B2 (obstacle radius 1.0, start$=[1,1]$, goal$=[17,17]$), running 20 trials per planner with the same B2 parameters. This scenario has tight passages where detours are costly, so shortcutting should make a visible difference.

RRT benefits the most: 14.8% shorter and 91% fewer waypoints, since its many small steps are easy to collapse. PRM improves less in length (−2.0%) since A* already gives near-optimal graph paths, but drops 69% of waypoints. After

	Success	Len Before	Len After	Len Δ	WP Δ	Plan [s]	Total [s]
PRM	100%	24.50 m	24.03 m	−2.0%	−69%	1.714	1.868
RRT	100%	28.72 m	24.46 m	−14.8%	−91%	0.068	0.171

Table 4: Shortcutting results (20 trials). Plan = planning time only. Total = planning + shortcutting.

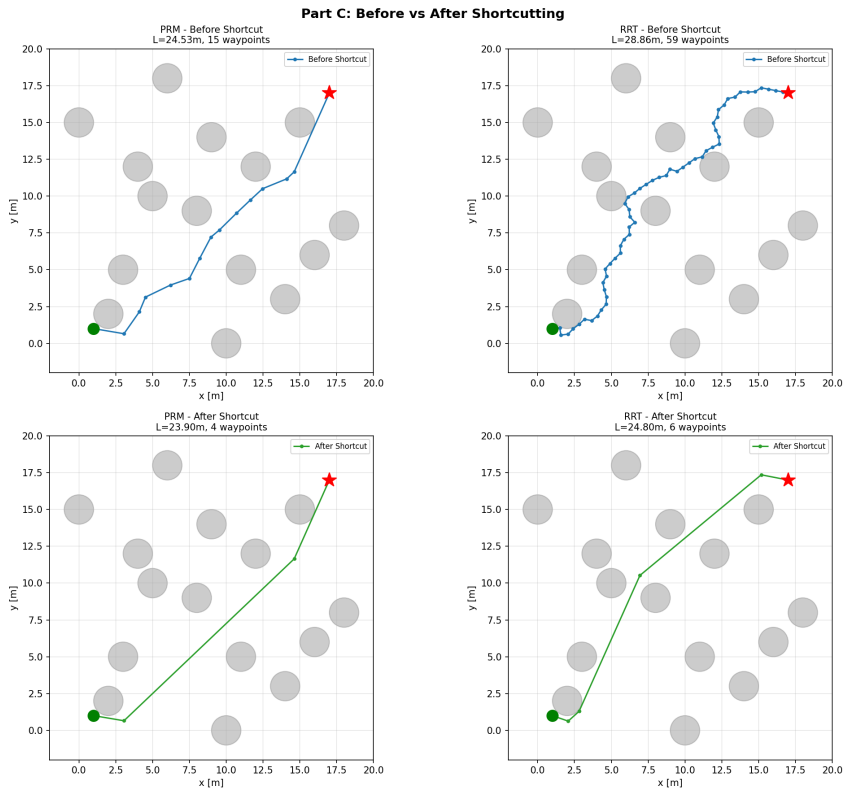


Figure 6: Before (blue) vs. after (green) shortcutting. RRT (right) shows dramatic simplification.

shortcutting, both converge to ∼24 m, likely near the true optimum.

Shortcutting adds only 0.15 s (PRM) and 0.10 s (RRT) to the total time. RRT + shortcutting takes 0.171 s total and produces 24.46 m paths, while PRM alone takes 1.714 s for 24.50 m paths. So RRT + shortcutting matches PRM quality at 10× lower cost. This eliminates RRT's main weakness (path quality) while preserving its strength (speed).

Use of AI Tools. I used Claude Code to write experiment scripts and plotting code. I designed the experiments, verified outputs, and wrote all analysis myself.